

処理型に関するレポート

1章 字句解析

字句解析を行うのは、`get_token()` 関数が実装されている。Rubyの標準ライブラリである`strscan`ライブラリを使い正規表現による字句解析を行う。

`get_token()` 関数は呼び出されると、数値、予約語、関数名、演算子などのトークンを識別し、対応するシンボルを返す役割を持っている。正規表現によって取得される主なトークンは以下の通りである。

- ①`val`にて、数値の判定を行う。
- ②`sys`にて、`FermaQED`の予約語の判定を行う。
- ③`func`にて、`FermaQED`の比較関数と関数の判定を行う。
- ④`op`にて、演算子の判定を行う。

、、、

```
def get_token()
  keywords = {
    # 計算用演算子
    '+' => :add,
    '-' => :sub,
    '*' => :mul,
    '/' => :div,
    '(' => :left_parn,
    ')' => :right_parn,
    '[' => :left_array,
    ']' => :right_array,
    '=' => :substitution,
    ':' => :colon,
    ',' => :comma,

    # 予約語
    'if' => :if,
    'elif' => :elif,
    'else' => :else,
    'while' => :while,
    'QED' => :QED,

    # 比較用関数
    'large_comparison(' => :def_big,
    'small_comparison(' => :def_small,
    'equal_comparison(' => :def_equal,

    # 関数
    'print(' => :def_print,
    'Ferma(' => :def_Ferma
  }

  @scanner.skip(/\s+/)
```

```

@scanner.skip(/\t+/)

if val = @scanner.scan(/\d+/)
  return val
end

if sys = @scanner.scan(/\b(if|elif|else|while|QED)\b/)
  return keywords[sys]
end

if func =
@scanner.scan(/print\(|large_comparison\(|small_comparison\(|equal
_comparison\(|Ferma\(/)
  return keywords[func]
end

if op = @scanner.scan(/[+\-\\*\\/\(\)\[\]\=\:\,\,]/)
  return keywords[op]
end
end
``,``,`

```

2章 構文解析

構文解析である関数、`parse()`関数は、以下の生成規則を元に動作を行う。生成規則に従い抽象構文木を作成する仕組みになっている。

①生成規則

- ・ 文列 = 文(文)*
- ・ 文 = 配列変数 `=` `式` QED | if 関数 `:` `文` (elif 関数 `:` `文`)* (else `:` `文`)? QED | while 関数 `:` `文` QED | print `( `式` )` QED | Ferma `(``)` QED
- ・ 関数 = 大比較 `( `式` , `式` )` | 小比較 `( `式` , `式` )` | 等しい `( `式` , `式` )`
- ・ 式 = 項 (`( `+` | `−`) 項) *
- ・ 項 = 因子 (`( `*` | `/`) 因子) *
- ・ 因子 = リテラル | 配列変数 | `( `式` )`
- ・ 配列変数 = `[ `リテラル` ]`

`parse()`関数は、与えられたコードを解析し、最終的に抽象構文木である配列を返す仕組みになっている。抽象構文木作成のための生成規則は、以下の関数で行われる。

① `sentence()` は文列を作成する。最終的に `[文A, 文B]` のように返す

```

``,``,`
def sentence()
  codes = []
  until @scanner.eos?
    if @debug == true
      puts "p2"
    end

    @scanner.skip(/\s+/)

```

```

        if @scanner.eos?
            break
        end

        codes.append(literature())
    end
    return codes
end
``,`

```

②literature()は文を作成する。

代入文は[シンボル, 変数名, 式]のように返す

if文は[シンボル, [関数, 式], 文]のように返す

while文は[シンボル, [関数, 文]]のように返す

print文は[シンボル, 式]のように返す

Ferma文は[シンボル, nil]のように返す

```

``,`
def literature()
    token = get_token()
    if token == :left_array
        if @debug == true
            puts "p3_1"
        end

        token = get_token()
        unless token.to_s =~ /\A\d+\z/
            raise Exception, "構文エラー"
        end
        index = token.to_i
        t = get_token()
        unless t == :right_array
            raise Exception, "構文エラー"
        end
        t = get_token()
        unless t == :substitution
            raise Exception, "構文エラー"
        end
        formula = expression()
        t = get_token()
        unless t == :QED
            raise Exception, "終端エラー"
        end
        result = [:substitution_writings, index, formula]

    elsif token == :if
        if @debug == true

```

```

    puts "p3_2"
end

box = [:if_writings]
func = func_parse()
t = get_token()
unless t == :colon
    raise Exception, "構文エラー"
end
formula = literature()
box.append([func, formula])
token = get_token()

while token == :elif
    if @debug == true
        puts "p3_2_1"
    end

    func = func_parse()
    t = get_token()
    unless t == :colon
        raise Exception, "構文エラー"
    end
    formula = literature()
    box.append([func, formula])
    token = get_token()
end

if token == :else
    if @debug == true
        puts "p3_2_2"
    end

    t = get_token()
    unless t == :colon
        raise Exception, "構文エラー"
    end
    formula = literature()
    box.append([[:elnil], formula])
    token = get_token()
end

unless token == :QED
    raise Exception, "終端エラー"
end
result = box

```

```

elsif token == :while
  if @debug == true
    puts "p3_3"
  end

  func = func_parse()
  t = get_token()
  unless t == :colon
    raise Exception, "構文エラー"
  end
  formula = literature()
  result = [:while_writings, func, formula]
  token = get_token()
  unless token == :QED
    raise Exception, "終端エラー"
  end
end

elsif token == :def_print
  if @debug == true
    puts "p3_4"
  end

  formula = expression()
  token = get_token()
  unless token == :right_parn
    raise Exception, "終端エラー"
  end
  token = get_token()
  unless token == :QED
    raise Exception, "終端エラー"
  end
  result = [:def_print_writings, formula]

elsif token == :def_Ferma
  if @debug == true
    puts "p3_4"
  end

  token = get_token()
  unless token == :right_parn
    raise Exception, "終端エラー"
  end
  token = get_token()
  unless token == :QED
    raise Exception, "終端エラー"
  end
  result = [:def_Ferma_writings, formula]

```

```

else
  # 式
  if @debug == true
    puts "p3_0"
  end

  unget_token(token)
  result = [:expression_writings, expression()]
  t = get_token()
  unless t == :QED
    raise Exception, "終端エラー"
  end
end

return result
end
` ``

```

③func_parse() は文を作成する。

大比較であるlarge_comparison文は[シンボル, xの式, yの式]のように返す

小比較であるsmall_comparison文は[シンボル, xの式, yの式]のように返す

等しいであるequal_comparison文は[シンボル, xの式, yの式]のように返す

```

` ``
def func_parse()
  token = get_token()
  if token == :def_big or token == :def_small or token ==
:def_equal
    if @debug == true
      puts "p4_1"
    end

    x_num = expression()
    t = get_token()
    unless t == :comma
      raise Exception, "構文エラー"
    end
    y_num = expression()
    t = get_token()
    unless t == :right_parn
      raise Exception, "構文エラー"
    end
    result = [token, x_num, y_num]
  end
  return result
end
` ``

```

④expression() は式、term() は項を作る。Calcと同じ動作をする。

```
def expression()
  if @debug == true
    puts "p5"
  end

  result = term()
  token = get_token()
  while token == :add or token == :sub
    result = [token, result, term()]
    token = get_token()
  end
  # puts "[expression] = #{result.inspect}"
  unget_token(token)
  return result
end

def term()
  if @debug == true
    puts "p6"
  end

  result = factor()
  token = get_token()
  while token == :mul or token == :div
    result = [token, result, factor()]
    token = get_token()
  end

  # puts "[term] = #{result.inspect}"

  unget_token(token)
  return result
end
```

⑤factor() は基本的にCalcと同じ動作をする。しかし、追加として配列変数が追加している。

最終的に式であるexpression() を呼ぶか、値、配列変数を呼ぶ。

```
def factor()
  if @debug == true
    puts "p7"
  end

  token = get_token()
```

```

if token.to_s =~ /\A\d+\z/
  if @debug == true
    puts "p7_1"
  end

  # puts "[factor] = #{token}"
  result = token.to_i
elsif token == :left_parn
  # puts "[factor] ("
  result = expression()
  t = get_token()
  unless t == :right_parn
    raise Exception, "構文エラー"
  end
end
elsif token == :left_array
  # puts "[factor] ("
  token = get_token()
  result = [token.to_i]
  t = get_token()
  unless t == :right_array
    raise Exception, "構文エラー"
  end
end
else
  raise Exception, "構文エラー"
end
return result
end
...

```

3章 解釈実行

構文木として生成した配列を解釈し、コードを実行する仕組みとなっている。

`eval_mein()` は文列を1つ1つループで実行する。そして、シンボルに対応した文を呼び出し実行する。

`eval` は主に計算をメインに行い、式、項、因子を解析し計算を行う。基本的に `Calc` と同じ動作をする。

`if_eval` は `if` 文専用の `eval` 関数。 `while_eval` は `while` 文専用の `eval` 関数となっている。

```

...

# 意味解析
def eval_mein(exp)
  exp.each_with_index do |value, index|
    if @debug_evsl == true
      @x = @x + 1
      puts @x
      print exp[index]
      puts
    end
  end
end

```



```

# puts exp[index][0]
# 文解析
case exp[index][0]
when :substitution_writings
  var = exp[index][1].to_s
  num = eval(exp[index][2])
  @arry_hash[var] = num

when :if_writings
  # print exp[index][1]
  if_eval(exp[index])

when :while_writings
  while_eval(exp[index])

when :def_print_writings
  num = eval(exp[index][1])
  puts num

when :def_Ferma_writings
  print @arry_hash
  puts ""

when :expression_writings
  num = eval(exp[index][1])
end
end

end

# 計算用_eval
def eval(exp)
  if exp.is_a?(Array) && exp.length == 1
    exp = @arry_hash[exp[0].to_s]
  end

  if exp.instance_of?(Array)
    case exp[0]
    when :add
      return eval(exp[1]) + eval(exp[2])
    when :sub
      return eval(exp[1]) - eval(exp[2])
    when :mul
      return eval(exp[1]) * eval(exp[2])
    when :div
      return eval(exp[1]) / eval(exp[2])
    end
  end
end

```

```

        end
    else
        return exp
    end
end

# if文_eval
def if_eval(exp)
    exp.shift
    bool = false

    exp.each_with_index do |value, index|
        case exp[index][0][0]
        when :def_big
            bool = func_big(exp[index][0])
        when :def_small
            bool = func_small(exp[index][0])
        when :def_equal
            bool = func_equal(exp[index][0])
        when :elnil
            bool = true
        end

        if bool == true
            # print exp[index][1]
            eval_mein([exp[index][1]])
            break
        end
    end
end
end

```

```

# while文_eval
def while_eval(exp)
    exp.shift
    bool = false

    case exp[0][0]
    when :def_big
        bool = func_big(exp[0])
    when :def_small
        bool = func_small(exp[0])
    when :def_equal
        bool = func_equal(exp[0])
    end

    while bool == true

```

```

        eval_mein([exp[1]])
        case exp[0][0]
        when :def_big
            bool = func_big(exp[0])
        when :def_small
            bool = func_small(exp[0])
        when :def_equal
            bool = func_equal(exp[0])
        end
    end
end

end

```

各種関数処理

```

def func_big(exp)
    x1 = eval(exp[1])
    x2 = eval(exp[2])
    if x1 > x2
        bool = true
    else
        bool = false
    end
    return bool
end

```

```

def func_small(exp)
    x1 = eval(exp[1])
    x2 = eval(exp[2])
    if x1 < x2
        bool = true
    else
        bool = false
    end
    return bool
end

```

```

def func_equal(exp)
    x1 = eval(exp[1])
    x2 = eval(exp[2])
    if x1 == x2
        bool = true
    else
        bool = false
    end
    return bool
end

```

、、、

4章 考察

チュートリアルとなるプログラムを実行し、全て正しく実行することができるものになっている。そして、FermaQEDは文字列の使用を禁止している言語であり、フェルマの名を冠するため全て数字または配列のみで動く言語として実装し正しく実装することができた。

5章 プログラム全体

、、、

```
class FermaQED

  require 'strscan'
  def initialize
    @debug = false
    @debug_evsl = false
    @x = 0

    # 初期化
    @arry_hash = {}

    # コードの取得
    if ARGV[0]
      # ファイルが存在しない例外処理
      begin
        @file_name = ARGV[0]
        file = open(@file_name)
        exp = file.read
        file.close
      rescue
        puts $!.to_s
        exit
      end
    end

    @scanner = StringScanner.new(exp)

    tree = parse()

    if @debug == true
      print tree
      puts "意味解析スタート"
    end
  end
end
```

```
    eval_mein(tree)
end
```

```
keywords = {
  '+' => :add,
  '-' => :sub,
  '*' => :mul,
  '/' => :div,
  '(' => :left_parn,
  ')' => :right_parn
}
```

意味解析

```
def eval_mein(exp)
  exp.each_with_index do |value, index|
    if @debug_evsl == true
      @x = @x + 1
      puts @x
      print exp[index]
      puts
    end
  end
end
```

```
# puts exp[index][0]
```

文解析

```
case exp[index][0]
when :substitution_writings
  var = exp[index][1].to_s
  num = eval(exp[index][2])
  @arry_hash[var] = num
end
```

```
when :if_writings
  # print exp[index][1]
  if_eval(exp[index])
end
```

```
when :while_writings
  while_eval(exp[index])
end
```

```
when :def_print_writings
  num = eval(exp[index][1])
  puts num
end
```

```
when :def_Ferma_writings
  print @arry_hash
  puts ""
end
```

```
when :expression_writings
end
```

```

        num = eval(exp[index][1])
    end

end

end

# 計算用_eval
def eval(exp)
    if exp.is_a?(Array) && exp.length == 1
        exp = @arry_hash[exp[0].to_s]
    end

    if exp.instance_of?(Array)
        case exp[0]
        when :add
            return eval(exp[1]) + eval(exp[2])
        when :sub
            return eval(exp[1]) - eval(exp[2])
        when :mul
            return eval(exp[1]) * eval(exp[2])
        when :div
            return eval(exp[1]) / eval(exp[2])
        end
    else
        return exp
    end
end

# if文_eval
def if_eval(exp)
    exp.shift
    bool = false

    exp.each_with_index do |value, index|
        case exp[index][0][0]
        when :def_big
            bool = func_big(exp[index][0])
        when :def_small
            bool = func_small(exp[index][0])
        when :def_equal
            bool = func_equal(exp[index][0])
        when :elnil
            bool = true
        end
    end
end

```

```

        if bool == true
            # print exp[index][1]
            eval_mein([exp[index][1]])
            break
        end
    end
end
end

```

while文_eval

```
def while_eval(exp)
```

```
    exp.shift
```

```
    bool = false
```

```

    case exp[0][0]
    when :def_big

```

```
        bool = func_big(exp[0])
```

```
    when :def_small
```

```
        bool = func_small(exp[0])
```

```
    when :def_equal
```

```
        bool = func_equal(exp[0])
```

```
    end
```

```

    while bool == true
        eval_mein([exp[1]])

```

```
        case exp[0][0]
```

```
        when :def_big
```

```
            bool = func_big(exp[0])
```

```
        when :def_small
```

```
            bool = func_small(exp[0])
```

```
        when :def_equal
```

```
            bool = func_equal(exp[0])
```

```
        end
```

```
    end
```

```
end
```

各種関数処理

```
def func_big(exp)
```

```
    x1 = eval(exp[1])
```

```
    x2 = eval(exp[2])
```

```
    if x1 > x2
```

```
        bool = true
```

```
    else
```

```
        bool = false
```

```
    end
```

```

    return bool
end

def func_small(exp)
    x1 = eval(exp[1])
    x2 = eval(exp[2])
    if x1 < x2
        bool = true
    else
        bool = false
    end
    return bool
end

```

```

def func_equal(exp)
    x1 = eval(exp[1])
    x2 = eval(exp[2])
    if x1 == x2
        bool = true
    else
        bool = false
    end
    return bool
end

```

構文解析器

```

def parse()
    if @debug == true
        puts "p1"
    end
    sentence()
end

def sentence()
    codes = []
    until @scanner.eos?
        if @debug == true
            puts "p2"
        end

        @scanner.skip(/\s+/)
        if @scanner.eos?
            break
        end
    end
end

```



```

        codes.append(literature())
    end
    return codes
end

def literature()
    token = get_token()
    if token == :left_array
        if @debug == true
            puts "p3_1"
        end

        token = get_token()
        unless token.to_s =~ /\A\d+\z/
            raise Exception, "構文エラー"
        end
        index = token.to_i
        t = get_token()
        unless t == :right_array
            raise Exception, "構文エラー"
        end
        t = get_token()
        unless t == :substitution
            raise Exception, "構文エラー"
        end
        formula = expression()
        t = get_token()
        unless t == :QED
            raise Exception, "終端エラー"
        end
        result = [:substitution_writings, index, formula]

    elsif token == :if
        if @debug == true
            puts "p3_2"
        end

        box = [:if_writings]
        func = func_parse()
        t = get_token()
        unless t == :colon
            raise Exception, "構文エラー"
        end
        formula = literature()
        box.append([func, formula])
        token = get_token()
    end
end

```

```

while token == :elif
  if @debug == true
    puts "p3_2_1"
  end

  func = func_parse()
  t = get_token()
  unless t == :colon
    raise Exception, "構文エラー"
  end
  formula = literature()
  box.append([func, formula])
  token = get_token()
end

if token == :else
  if @debug == true
    puts "p3_2_2"
  end

  t = get_token()
  unless t == :colon
    raise Exception, "構文エラー"
  end
  formula = literature()
  box.append([[:elnil], formula])
  token = get_token()
end

unless token == :QED
  raise Exception, "終端エラー"
end
result = box

elsif token == :while
  if @debug == true
    puts "p3_3"
  end

  func = func_parse()
  t = get_token()
  unless t == :colon
    raise Exception, "構文エラー"
  end
  formula = literature()
  result = [:while_writings, func, formula]
  token = get_token()
end

```

```

        unless token == :QED
          raise Exception, "終端エラー"
        end

      elsif token == :def_print
        if @debug == true
          puts "p3_4"
        end

        formula = expression()
        token = get_token()
        unless token == :right_parn
          raise Exception, "終端エラー"
        end
        token = get_token()
        unless token == :QED
          raise Exception, "終端エラー"
        end
        result = [:def_print_writings, formula]

      elsif token == :def_Ferma
        if @debug == true
          puts "p3_4"
        end

        token = get_token()
        unless token == :right_parn
          raise Exception, "終端エラー"
        end
        token = get_token()
        unless token == :QED
          raise Exception, "終端エラー"
        end
        result = [:def_Ferma_writings, formula]

      else
        # 式
        if @debug == true
          puts "p3_0"
        end

        unget_token(token)
        result = [:expression_writings, expression()]
        t = get_token()
        unless t == :QED
          raise Exception, "終端エラー"
        end
      end
    end
  end
end

```

```

end

return result
end

def func_parse()
  token = get_token()
  if token == :def_big or token == :def_small or token ==
:def_equal
    if @debug == true
      puts "p4_1"
    end

    x_num = expression()
    t = get_token()
    unless t == :comma
      raise Exception, "構文エラー"
    end
    y_num = expression()
    t = get_token()
    unless t == :right_parn
      raise Exception, "構文エラー"
    end
    result = [token, x_num, y_num]
  end
  return result
end

def expression()
  if @debug == true
    puts "p5"
  end

  result = term()
  token = get_token()
  while token == :add or token == :sub
    result = [token, result, term()]
    token = get_token()
  end
  # puts "[expression] = #{result.inspect}"
  unget_token(token)
  return result
end

def term()
  if @debug == true

```

```

    puts "p6"
  end

  result = factor()
  token = get_token()
  while token == :mul or token == :div
    result = [token, result, factor()]
    token = get_token()
  end

  # puts "[term] = #{result.inspect}"

  unget_token(token)
  return result
end

def factor()
  if @debug == true
    puts "p7"
  end

  token = get_token()
  if token.to_s =~ /\A\d+\z/
    if @debug == true
      puts "p7_1"
    end

    # puts "[factor] = #{token}"
    result = token.to_i
  elsif token == :left_parn
    # puts "[factor] ("
    result = expression()
    t = get_token()
    unless t == :right_parn
      raise Exception, "構文エラー"
    end
  elsif token == :left_array
    # puts "[factor] ("
    token = get_token()
    result = [token.to_i]
    t = get_token()
    unless t == :right_array
      raise Exception, "構文エラー"
    end
  else
    raise Exception, "構文エラー"
  end
end

```

```
    return result
end
```

```
def get_token()
  keywords = {
    # 計算用演算子
    '+' => :add,
    '-' => :sub,
    '*' => :mul,
    '/' => :div,
    '(' => :left_parn,
    ')' => :right_parn,
    '[' => :left_array,
    ']' => :right_array,
    '=' => :substitution,
    ':' => :colon,
    ',' => :comma,

    # 予約語
    'if' => :if,
    'elif' => :elif,
    'else' => :else,
    'while' => :while,
    'QED' => :QED,

    # 比較用関数
    'large_comparison(' => :def_big,
    'small_comparison(' => :def_small,
    'equal_comparison(' => :def_equal,

    # 関数
    'print(' => :def_print,
    'Ferma(' => :def_Ferma
  }

  @scanner.skip(/\s+/)
  @scanner.skip(/\t+/)

  if val = @scanner.scan(/\d+/)
    return val
  end

  if sys = @scanner.scan(/\b(if|elif|else|while|QED)\b/)
    return keywords[sys]
  end
end
```

```

    if func =
@scanner.scan(/print\(|large_comparison\(|small_comparison\(|equal
_comparison\(|Ferma\(/)
    return keywords[func]
end

    if op = @scanner.scan(/[\+\-\*\\/\(\)\[\]\=\:\,\]/)
    return keywords[op]
end
end

# 触れない
def unget_token(token)
  begin
    @scanner.unscan
  rescue
  end
end

end

fermaQED = FermaQED.new

...

```

6章 簡易テスト結果

count.ferma	出力結果
[0] = 1QED print([0])QED	1
print(1+1)QED	2
[0] = 1QED [1] = [0]QED [3] = 3QED [2] = 1+2QED [4] = [0] + [0]QED Ferma()QED	{ "0"=>1, "1"=>1, "3"=>3, "2"=>3, "4"=>2 }
[5] = 10QED print([5])QED while large_comparison([5],1): [5]= [5] - 1QED QED Ferma()QED	10 { "5"=>1 }
if large_comparison(2,1):	1

print(1) QED QED	
if large_comparison(1,2): print(2) QED QED	
if small_comparison(1,2): print(3) QED QED	3
if small_comparison(2,1): print(4) QED QED	
if equal_comparison(2,1): print(5) QED QED	
if equal_comparison(1,1): print(6) QED QED	6
if large_comparison(2,2): print(7) QED elif large_comparison(2,1): print(8) QED QED	8
if large_comparison(2,2): print(14) QED elif large_comparison(2,2): print(15) QED elif large_comparison(2,1): print(16) QED QED	16
if large_comparison(2,2): print(17) QED elif large_comparison(2,2): print(18) QED elif large_comparison(2,2): print(19) QED else : print(20) QED QED	20
if large_comparison(2,1): if large_comparison(2,1): print(21) QED QED QED	21